

HPC, Cloud: Algoritmos y Paralelización

Índice de contenido

Introducción.....	1
Algoritmos paralelizables y no paralelizables.....	1
Diferentes arquitecturas.....	2
La Ecuación del Calor.....	2
Discretización.....	4
Discretización de la ecuación del calor.....	4
Nota metodológica.....	5
Visualización gráfica del algoritmo.....	6
Código para Algoritmo Secuencial.....	7
Complejidad del algoritmo.....	7
Ejecución en un procesador (modo no paralelo).....	8
Definición de eficiencia paralela.....	8
Paralelismo en modo Granja, Compulsivo o Trivial.....	8
Eficiencia en el caso de paralelismo compulsivo.....	9
Paralelismo débil.....	9
Complejidad y memoria por procesador.....	10
Comunicación entre procesadores.....	10
Nota.....	12
Eficiencia.....	12
Paralelismo fuerte, Paralelismo masivo, Procesadores acoplados.....	12
Propuesta de Ejercicio.....	13
Un ejemplo más realista.....	13
Ejemplo de código paralelo usando threads.....	14

Introducción

Este archivo puede descargarse en

<https://drive.google.com/open?id=1xSPvCIMZeHn0E2TweGgidn48hye5H52d>

Los algoritmos numéricos, simulaciones, cálculos en general, adquieren cada día mayor complejidad y se requiere más y más potencia de cálculo y memoria para su resolución. Una forma de avanzar es hacer que diferentes procesadores trabajen en el mismo problema de modo que logramos obtener la solución en tiempos menores.

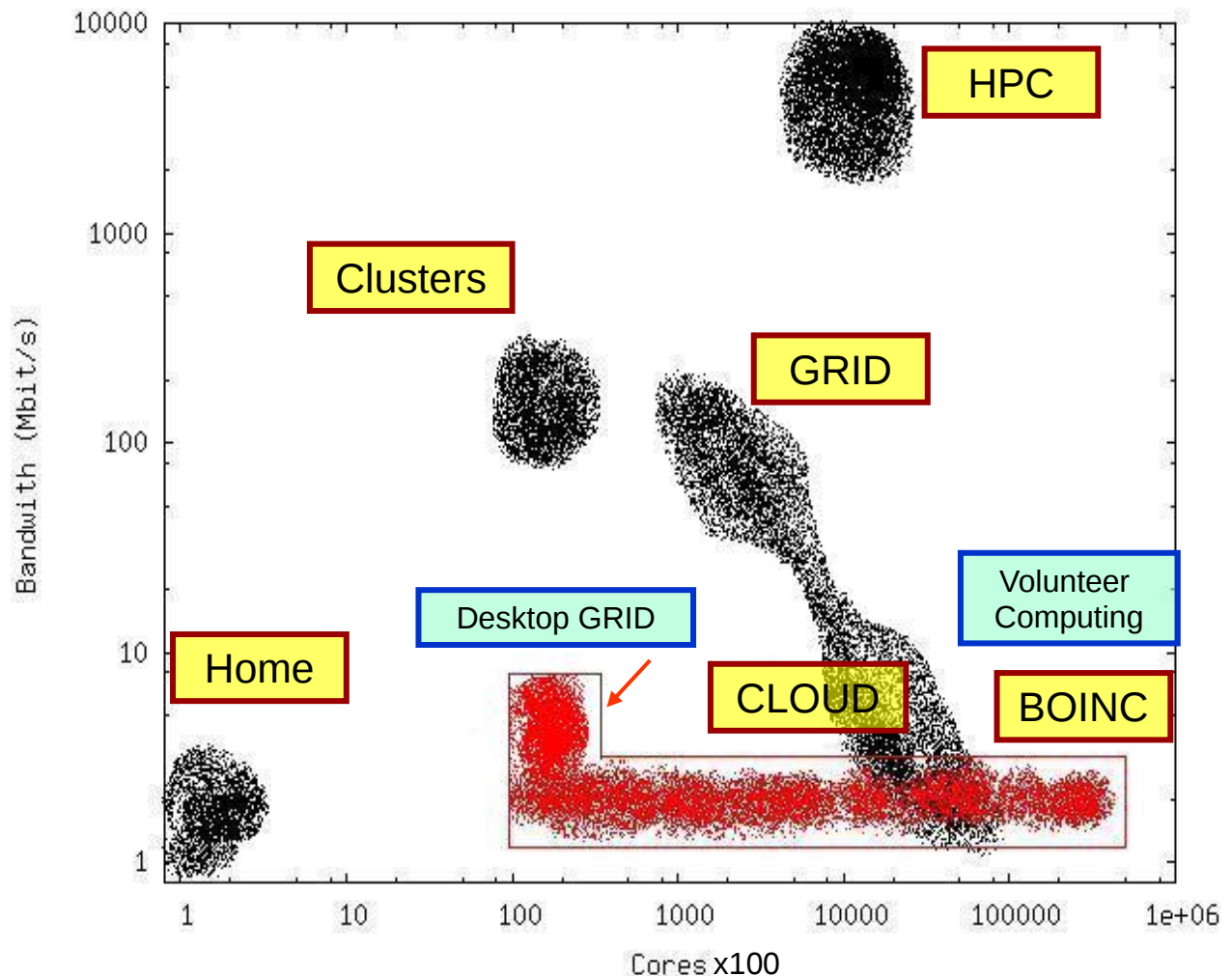
El proceso de reescribir un código, en general ya conocido y usado en un simple procesador, de forma que se ejecute en varios, es lo que llamamos Paralelización. Esto es normalmente un proceso bastante complejo, existiendo algoritmos que ni tan siquiera son paralelizables.

Algoritmos paralelizables y no paralelizables

No todas las tareas son segmentables en tareas más simples para reducir el tiempo de ejecución. Puede ocurrir que el problema sea tal que no admita su ejecución eficiente en varios procesadores. Aquí estudiaremos un ejemplo de algoritmo paralelizable que nos sirva de soporte para discutir los conceptos e ideas principales.

Diferentes arquitecturas

Existen diferentes tipos de arquitecturas y plataformas de computación. En la gráfica adjunta se puede ver el número de núcleos (o cores) de cálculo típicos para cada una de ellas y el ancho de banda, dos de los parámetros relevantes en la subsiguiente discusión.



La Ecuación del Calor

Par ilustrar el problema, utilizaremos un fenómeno físico bien conocido y estudiado (originalmente por Newton), que tiene importantes connotaciones tecnológicas y que es sencillo desde el punto de vista conceptual y numérico: la propagación del calor.

Dado un cierto material, cada uno de sus puntos está a una temperatura dada en un

momento inicial. Si suponemos conocidas todas las características del material, la pregunta es ¿Cómo cambia la temperatura del material con el tiempo?

Para materiales homogéneos, la física del problema es enormemente simple, y obedece a la siguiente regla:

La temperatura de cada punto varía proporcionalmente al Laplaciano en dicho punto.

Como una ecuación, toma esta forma

$$\frac{\partial T(\vec{r}, t)}{\partial t} = k \left(\frac{\partial^2 T(\vec{r}, t)}{\partial x^2} + \frac{\partial^2 T(\vec{r}, t)}{\partial y^2} + \frac{\partial^2 T(\vec{r}, t)}{\partial z^2} \right)$$

Veamos algunas propiedades elementales, para comprender mejor el problema físico, comprensión (siempre) necesaria para comenzar a escribir código, y paralelizarlo posteriormente.

Dada una situación inicial, es decir, dada la temperatura en todos los puntos del material en un instante fijo, el calor fluye de las partes calientes a las frías, es decir, las partes rodeadas de puntos más calientes, se calientan, las partes rodeadas de punto más fríos, se enfrían.

Si suponemos unos cuantos puntos (los bordes por ejemplo) a una temperatura fija, el sistema tiende a una solución estacionaria. Por ejemplo, supongamos una barra de longitud R , con un extremo a la temperatura T_0 y el otro a la temperatura T_1 . En el caso de una varilla, la ecuación anterior toma la forma

$$\frac{\partial T(\vec{r}, t)}{\partial t} = k \frac{\partial^2 T(\vec{r}, t)}{\partial x^2}$$

En este caso, si dejamos pasar mucho tiempo, la Temperatura será constante en cada punto: no variará con el tiempo, con lo cual tendremos para la solución estacionaria,

$$\frac{\partial T(x, t)}{\partial t} = 0 = k \frac{\partial^2 T(x, t)}{\partial x^2}$$

Entonces la solución es una temperatura con derivada segunda nula. La solución en este caso, teniendo en cuenta que tenemos los extremos a Temperaturas fijas, es

$$T(x, t) = T(x_0) + \frac{x}{R} (T(x_1) - T(x_0))$$

es decir, la temperatura va cambiando linealmente desde un extremo a otro, y esa es una solución que se mantiene en el tiempo. Esto se puede comprobar inmediatamente: la derivada segunda de una recta es nula.

- ¿Cuál es el significado físico de k ? ¿Qué unidades tiene?
- ¿Por qué la solución estacionaria no depende de k ? ¿Qué depende de k ?

Discretización

En el ordenador habitualmente se pasa a trabajar en lugar de con el espacio y tiempo continuos, con espacio y tiempos discretos. Es decir, el material es sustituido por un conjunto de puntos distribuidos en el espacio: en lugar de tener un continuo tenemos una red más o menos homogénea. Por ejemplo, supongamos que el material a estudiar es una varilla muy fina, entonces sustituimos dicha varilla, por un conjunto de L puntos $(n_0, n_1, n_2, \dots, n_{L-1})$. Si la longitud de la varilla era R metros, ahora estos puntos estarán separados por una distancia $a=R/L$. Cuanto menor sea esta distancia, mejor es la aproximación de sustituir el continuo por el discreto.



También discretizamos el tiempo, de modo que ahora será $(t_0, t_1, t_2, \dots)$, y el intervalo entre cada valor será $\delta = t_{i+1} - t_i$

En esta forma discreta, la cantidad a estudiar, la Temperatura, es una función que depende de n_i , y que toma valores en general diferentes para cada valor del tiempo t_i , es decir

$$T(x, t) \rightarrow T(n_i, t_k)$$

Discretización de la ecuación del calor

Debemos ver ahora que forma toma la ecuación diferencial en términos de las variables discretas. Basta recordar que la derivada de una función $f(x)$, se puede aproximar bastante bien por

$$\frac{d f(x)}{d x} \approx \frac{f(x+h)-f(x)}{h}$$

siempre que h sea *suficientemente* pequeño.

Entonces, en nuestro caso, la derivada con respecto al tiempo será

$$\frac{\partial T(x, t)}{\partial t} \approx \frac{T(x, t+\delta)-T(x, t)}{\delta}$$

y tomando tiempos sólo en los puntos discretos, y sabiendo que $t_i + \delta = t_{i+1}$, tendremos

$$\frac{\partial T(x, t)}{\partial t} \equiv \frac{T(n_i, t_{k+1})-T(n_i, t_k)}{\delta}$$

Ahora debemos discretizar la derivada segunda espacial. Para ello basta recordar que la

derivada segunda es la derivada de una derivada. Para la primera derivada tendremos,

$$\frac{\partial T(x,t)}{\partial x} \equiv \frac{T(n_{i+1},t_k) - T(n_i,t_k)}{a}$$

Ahora debemos derivar de nuevo la expresión anterior. Tras unas pocas operaciones elementales, llegamos a

$$\frac{\partial^2 T(x,t)}{\partial^2 x} \equiv \frac{T(n_{i+1},t_k) + T(n_{i-1},t_k) - 2T(n_i,t_k)}{a^2}$$

Con todo ello la ecuación del calor discretizada es

$$\frac{T(n_i,t_{k+1}) - T(n_i,t_k)}{\delta} = k \frac{T(n_{i+1},t_k) + T(n_{i-1},t_k) - 2T(n_i,t_k)}{a^2}$$

o en otra forma

$$T(n_i,t_{k+1}) = T(n_i,t_k) + \frac{k\delta}{a^2} (T(n_{i+1},t_k) + T(n_{i-1},t_k) - 2T(n_i,t_k))$$

y agrupando los términos

$$T(n_i,t_{k+1}) = T(n_i,t_k) - 2\frac{k\delta}{a^2} T(n_i,t_k) + \frac{k\delta}{a^2} (T(n_{i+1},t_k) + T(n_{i-1},t_k))$$

o mejor aún

$$T(n_i,t_{k+1}) = (1 - 2\frac{k\delta}{a^2}) T(n_i,t_k) + \frac{k\delta}{a^2} (T(n_{i+1},t_k) + T(n_{i-1},t_k))$$

y finalmente

$$\boxed{T(n_i,t_{k+1}) = (1 - 2C) T(n_i,t_k) + C (T(n_{i+1},t_k) + T(n_{i-1},t_k))}$$

$$C = \frac{k\delta}{a^2}$$

En esta forma ya tenemos la solución y el algoritmo numérico para resolver el problema: dado el valor de la temperatura en todos los puntos de la varilla en un momento dado de tiempo, mediante la ecuación anterior podemos calcular el valor de la temperatura en todos los puntos en el instante de tiempo siguiente. Iterando el proceso, tenemos la solución completa al problema.

- **¿Cuáles son las unidades de C? ¿Como depende la llegada al equilibrio de k,δ,a?**

Nota metodológica

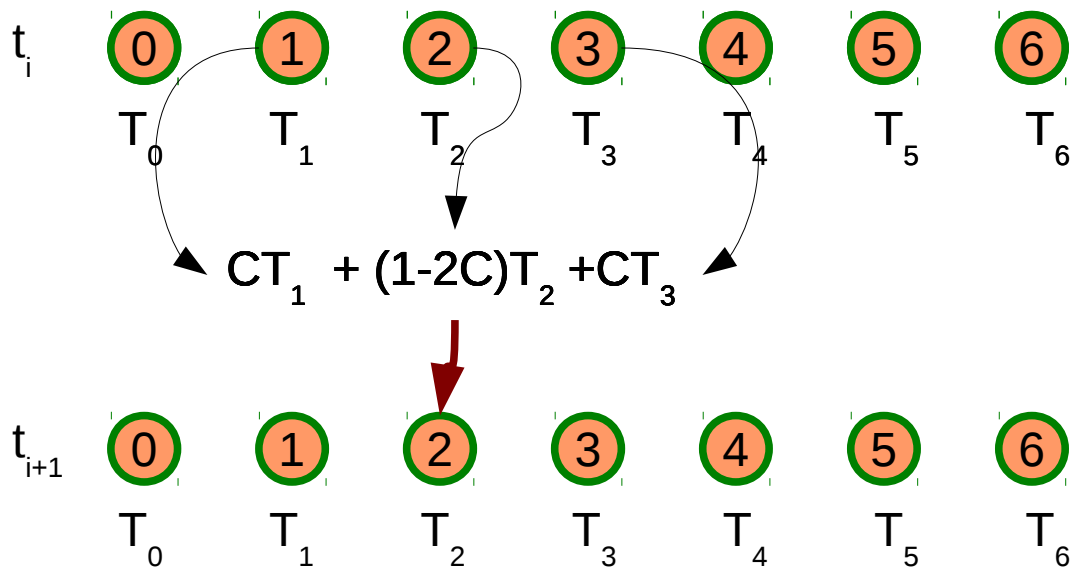
El algoritmo propuesto aquí para resolver este problema es muy poco eficiente, en el sentido de que es necesario un tiempo de CPU (o número de iteraciones) para resolverlo. Existen algoritmos mucho mas eficaces (leap-frog, Runge-Kutta, predictor-corrector..) pero son más complejos numérica y conceptualmente. Aquí usamos este algoritmo no para resolver realmente el problema sino simplemente para ilustrar el problema de la paralelización; dedicamos poco tiempo a la parte algorítmica para centrarnos en los aspectos de la paralelización.

Visualización gráfica del algoritmo

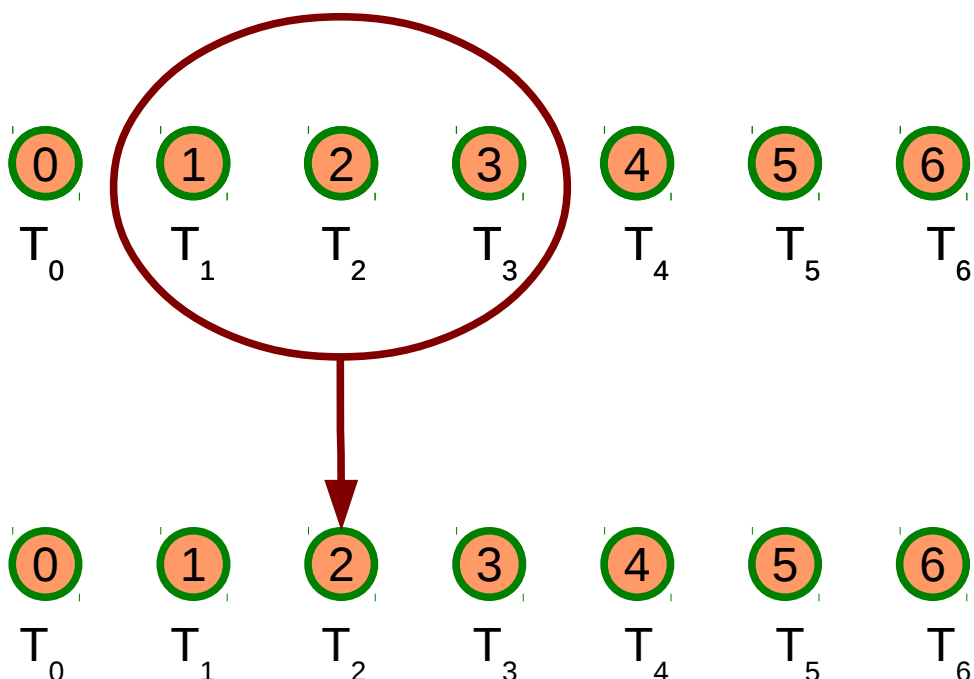
Recordemos la ecuación anterior,

$$T(n_i, t_{k+1}) = C_0 T(n_i, t_k) + C(T(n_{i+1}, t_k) + T(n_{i-1}, t_k)), C_0 = 1 - 2C$$

En cada momento debemos tener dos copias del sistema: el sistema en el momento anterior, y a partir de esos datos calculamos la temperatura en el momento siguiente. Remarcar que para calcular los nuevos datos usamos sólo los viejos: no usamos los que se van calculando a cada paso. Sólo cuando acabamos de cambiar todos los puntos, el sistema nuevo pasa a ser el viejo para la siguiente iteración.



Vemos que tenemos dos subsistemas diferentes, uno a tiempo t_i y otro a tiempo t_{i+1} . Para calcular la temperatura en un punto dado, necesitamos el valor de la temperatura en el momento anterior en el mismo punto y en los puntos vecinos, Más esquemáticamente,



Código para Algoritmo Secuencial

Durante el proceso trabajamos con dos copias del sistema. Una mantiene las variables en el momento *inicial*, la otra en el momento *final*. Cuando calculamos las variables en el momento final, para el siguiente paso, éstas pasan a ser la variables iniciales. Las constantes se calculan al principio una sola vez. Los puntos inicial y final no se cambian pues los suponemos a temperatura fija.

```
#define L 100000          // tamaño de la malla
double u[2*L];

for (i=0;i<100000;i+=2) {
    evolucion(u,u+L);
    evolucion(u+L,u);
}

double evolucion(double *uin, double *uout)
{
    int i;

    for (i=1;i<L-1;i++)
        uout[i]=C0*uin[i]+C*(uin[i-1]+uin[i+1]);
}
```

Complejidad del algoritmo

Para alcanzar la solución final, donde el sistema permanece estable pueden ser necesarias más o menos iteraciones dependiendo de la geometría, del número de puntos, de las constantes, etc. Por ello nos fijaremos en lo que cuesta una iteración.

Cada iteración debe realizar L pasos elementales, cada uno de los cuales es,

$$T(n_i, t_{k+1}) = C_0 T(n_i, t_k) + C (T(n_{i+1}, t_k) + T(n_{i-1}, t_k))$$

Vemos que cada paso elemental consta de 2 multiplicaciones y 2 sumas. Supongamos que las variables son en precisión simple (32 bits). Para realizar esto necesitamos 3 lecturas de la memoria RAM y una escritura, suponiendo que las constantes (C y C₀) están almacenadas en la caché o en registros del procesador. No consideraremos esos tiempos para nuestro análisis, pues si bien son relevantes, para los aspectos de esta charla, no son esenciales y añadirían bastante complejidad.

La memoria RAM necesaria para el problema son $L*4$ bytes para cada copia del sistema;

como tenemos dos de ellas, necesitaremos $L*8$ bytes.

Por tanto cada paso elemental consta de 4 operaciones en coma flotante. Cada iteración consta de L de esos pasos, de modo que la complejidad de una iteración es $4L$.

Ejecución en un procesador (modo no paralelo)

Un procesador realiza un cierto número de operaciones en coma flotante por segundo. Supongamos que es capaz de realizar una operación por ciclo, y que el procesador tiene una frecuencia de 2 Ghz (es decir, 1 ciclo será medio nanosegundo). El procesador tendrá una potencia de 2 Gflop por segundo, o 2Gflops. (nota: también se dice 2 Gflops por segundo...) Entonces, una iteración le costará un tiempo

$$\tau = \frac{4L}{2 \times 10^9} = 2L \times 10^{-9} \text{ seg.}$$

o lo que es lo mismo, el número de iteraciones por segundo, será,

$$\nu = \frac{1}{\tau} = \frac{1}{L} 5 \times 10^{-9} \text{ iter / seg}$$

Para un sistema con $L=100.000$, concluimos que podrá hacer del orden de 5000 iteraciones por segundo.

Definición de eficiencia paralela

Dado un problema, supongamos que al ejecutarlo en un solo procesador emplea un tiempo t . El caso ideal es que distribuyamos el trabajo entre N procesadores, de modo que el tiempo total sea t/N . La eficiencia paralela es el 100%. (más de esto es imposible: no confundir paralelizar con un milagro...).

En general, al dividir la tarea en N procesadores, el tiempo para resolver el problema será mayor de t/N . Si por ejemplo usáramos diez procesadores, pero el tiempo de ejecución no es diez veces menor, sino sólo 5 veces menor, diremos que la eficiencia es del 50%.

Podemos definir con precisión la eficiencia paralela (en %) como

$\epsilon = 100 \cdot \text{Tiempo empleado usando 1 procesador} / (N \cdot \text{tiempo empleado en cada procesador cuando usamos } N \text{ procesadores})$, o más esquemáticamente

$$\epsilon = 100 \frac{t_{1P}}{N t_{NP}}$$

Paralelismo en modo Granja, Compulsivo o Trivial

No es paralelismo en el sentido estricto. Imaginemos que tenemos que estudiar una barra con mil condiciones diferentes de temperatura en los extremos.

Estos son procesos independientes, que podríamos ejecutar uno detrás de otro en un sólo procesador, pues no hay información que transmitir de una ejecución a otra.

Entonces podemos enviar cada uno de los mil trabajos (jobs, runs, ejecuciones...) a mil procesadores diferentes, y cada uno trabajará de modo absolutamente independiente de los demás.

Eficiencia en el caso de paralelismo compulsivo

En el caso del paralelismo compulsivo, si tenemos por ejemplo 1000 procesos cada uno de los cuales ocupa un tiempo t , en un solo procesador necesitaremos un tiempo $1000t$. Si lo ejecutamos en 1000 procesadores, en cada uno de ellos el tiempo será t , y entonces para la eficiencia tendremos

$$\varepsilon = 100 * 1000t / (1000t) = 100\%$$

Esto tiene algunos matices que conviene remarcar. Como estamos viendo en el curso existen diferentes tipos de infraestructuras de cálculo: Ordenadores personales, Clusters, GRID, Super Ordenadores... Cada uno de estos sistemas tiene un comportamiento diferente.

En HPC, si enviamos 1000 Jobs independientes, se ejecutarán en cuanto tengamos acceso a las colas, todos los procesadores serán idénticos y todos acabarán a la vez. Esto es cierto en un cluster homogéneo y local (salvo que quizá no tengamos 1000 núcleos de cálculo disponibles).

En el GRID y otras infraestructuras distribuidas, la situación es algo más compleja.

Nosotros enviamos 1000 jobs. Las fuentes de incertidumbre son

- No todos los jobs comienzan a ejecutarse a la vez
- No todos los procesadores son iguales
- Existe una (no despreciable) probabilidad de perder jobs

Si el proceso sólo lo consideramos terminado cuando tenemos los 1000 jobs ejecutados, tenemos que contar como tiempo total el del último job recibido. Si estos tiempos finales de recepción se alargan mucho, la eficiencia puede disminuir estrepitosamente.

Paralelismo débil

Recordemos que en un sistema con 100.000 puntos, un procesador puede hacer del orden de 5000 iteraciones por segundo.

La cuestión es que para converger (llegar a una temperatura estacionaria) son necesarias un gran número de iteraciones supongamos que este caso es del orden de cien millones.

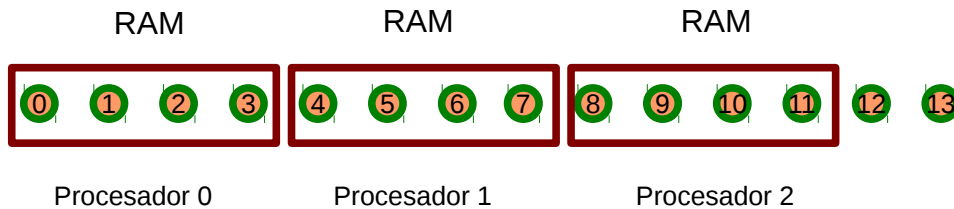
Esto hace que el tiempo de CPU necesario, o dicho de otra manera, el tiempo que debemos esperar a tener una solución, será del orden de

$$t = 100000000 / 5000 = 20000 \text{ segundos}$$

es decir, unas 5 horas y media.

Para acortar este tiempo, podemos repartir el trabajo entre varios procesadores.

El algoritmo que estamos usando es fácilmente paralelizable. Basta dividir la barra en M segmentos, y asignar un trozo a cada procesador,



Complejidad y memoria por procesador

Supongamos que tenemos L puntos, y N procesadores. Cada uno de ellos contendrá L/N puntos, y en cada iteración deberá realizar $4 \cdot L/N$ operaciones, y necesitará $8 \cdot L/N$ bytes de memoria RAM. Visto sólo esto, el tiempo quedaría dividido por N, pero no es así, pues el problema físico necesita de comunicación entre procesadores, que siempre supone una ralentización del proceso.

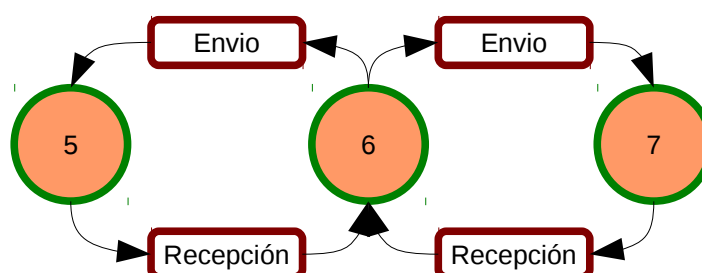
Comunicación entre procesadores

El paso elemental consiste en cambiar el valor de la temperatura, para lo cual hace falta saber el valor en el punto actual, el anterior y el posterior.

Supongamos que trabajamos con una máquina de memoria distribuida, donde cada procesador tiene su propia RAM, a la que solo puede acceder él, mientras que la comunicación de información entre procesadores diferentes se realiza a través de algun hardware específico, como por ejemplo Ethernet a 1 Gbit/seg, Infiniband, Myrinet...

En los puntos interiores del sistema, para saber estos valores leemos en la RAM del propio procesador, pero en los puntos de la frontera, el valor hacia adelante o hacia atrás *viven* en la RAM de otro procesador.

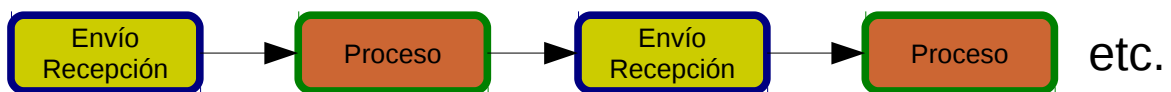
Debemos pedir pues este dato al procesador que lo posee. Llamaremos vecino de un procesador a aquel que tiene un punto del cual debe saber su valor para el proceso elemental. En nuestro ejemplo cada procesador tiene dos vecinos: uno hacia adelante y otro hacia atrás (excepto los extremos)



Supongamos que cada procesador puede comunicar directamente con sus vecinos. Esto implica cierta topología en el hardware de comunicación, y la suponemos disponible.

Una iteración global de todo el sistema, supone en cada procesador (o nodo) tres pasos fundamentales:

- Recibir información de sus vecinos de los valores necesarios (en este caso la Temperatura)
- Enviar a sus vecinos la información propia que les es necesaria
- Procesar toda la información y realizar una iteración en todas sus variables.



Esto requiere

- $4 \cdot L/N$ operaciones aritméticas
- Enviar 4 bytes a la Izquierda
- Enviar 4 bytes a la Derecha
- Recibir 4 bytes de la Izquierda
- Recibir 4 bytes de la derecha

En este ejemplo el orden es relevante y debe ser

1. Enviamos y recibimos datos
2. Procesamos

y el tiempo necesario (T) será la suma del tiempo de proceso más el de comunicación,

$$T = t_{\text{proceso}} + t_{\text{comunicacion}} \equiv \tau + \mu$$

Tiempo de Proceso (τ): Cada procesador hace $4 \cdot L/N$ operaciones aritméticas en coma flotante, el tiempo necesario será

$$\tau = \frac{4L}{N} \frac{1}{2 \cdot 10^9} = \frac{L}{N} 2 \times 10^{-9} \text{ seg.}$$

Tiempo de comunicación (μ): tiene dos partes

$$\mu = 2 \cdot t_{\text{latencia}} + t_{\text{transferencia}}$$

El tiempo de establecimiento de la comunicación, o tiempo de latencia, depende mucho del hardware (cuanto más caro, es menor...). En HPC está en torno a los 5 microsegundos, en otros sistemas como GRID puede ser milisegundos o superior; en una red domestica puede ser décimas o rozar el segundo.

La velocidad de transferencia también depende del hardware. En HPC puede ser del orden de 10 Gbits y superior, en los sistemas distribuidos varia entre los 100-10 Mbits.

Supongamos en nuestro problema que tenemos,

$$t_{latencia} = 5 \mu segs, v_{transferencia} = 100 Mbits$$

entonces

$$\mu = 2 * 5 * 10^{-6} + \frac{8 * 8}{100 * 10^6} = 10^{-5} + 6.4 * 10^{-7}$$

Nota

El proceso de cálculo y el proceso de comunicación algunas veces es posible solaparlos en parte (empiparlos) de modo que el tiempo total es algo mejor que la suma de ambos tiempos.

Eficiencia

El tiempo (en segundos) total de cada iteración en un procesador, será

$$T = \tau + \mu = \frac{L}{N} 2 \times 10^{-9} + 10^{-5} + 6.4 * 10^{-7}$$

El primer término decrece con el número de procesadores, pero no los dos segundos. Por lo tanto, mientras el primer término sea muy grande comparado con los otros, la paralelización será eficiente: el tiempo de espera decrece como 1/N, de modo que añadiendo más procesadores, disminuimos el tiempo.

Dicho de otro modo, la eficiencia, será

$$\epsilon = \frac{t_{1P}}{N * t_{NP}} = \frac{L \times 2 \times 10^{-9}}{L \times 2 \times 10^{-9} + N \times (10^{-5} + 6.4 * 10^{-7})} = \frac{1}{1 + \frac{N}{L} \times 5.3 \times 10^3}$$

Vemos que cuando $L \gg N$ la eficiencia es prácticamente 1. Si duplicamos el número de procesadores, el tiempo de espera se reduce a la mitad.

Sin embargo, si aumentamos el número de procesadores, de modo que N se acerca a L, la eficiencia disminuye. En el caso anterior, con $L=100.000$, si usamos $N=100$ procesadores (que es una cantidad modesta) tendremos una eficiencia de 0.16, el 16%, ya muy baja.

Paralelismo fuerte, Paralelismo masivo, Procesadores acoplados

Cuando nuestro problema necesita muchos procesadores, la cantidad de datos a transmitir es grande y con frecuencia, estamos ante un problema de paralelismo fuerte. Necesitamos un procesador potente, una red con baja latencia y alto ancho de banda. Si bien el ejemplo anterior es un poco pobre para este caso, los hechos básicos pueden extraerse del mismo, entendiendo siempre que los ejemplos son ilustrativos, en absoluto correspondientes con algo que se haría realmente para este caso.

Cuando el numero de procesadores crece mucho, el tiempo de cálculo y el tiempo de comunicación son comparables; incluso el de comunicación puede ser mayor. Si los procesadores no reciben la información de sus vecinos, deben permanecer parados, con la inevitable bajada de eficiencia.

Propuesta de Ejercicio

Podemos acercarnos aun problema con mayor paralelismo, considerando nuestro modelo, pero en tres dimensiones. Ahora la malla son puntos en una red cúbica, y podemos asignar a cada procesador un cubo más pequeño. La frontera ahora es mayor, y además depende del número de procesadores.

Es un excelente ejercicio realizar todos los cálculos anteriores para este caso.

Hay que prestar especial atención a la discretización de la ecuación diferencial. Cómo ayuda damos un resultado intermedio,

$$T(n_i, n_j, n_l, t_{k+1}) = T(n_i, n_j, n_l, t_k) + \frac{k \delta}{a^2} (\Delta_i + \Delta_j + \Delta_l)$$

$$\Delta_i = T(n_{i+1}, n_j, n_l, t_k) + T(n_{i-1}, n_j, n_l, t_k) - 2T(n_i, n_j, n_l, t_k)$$

$$\Delta_j = T(n_i, n_{j+1}, n_l, t_k) + T(n_i, n_{j-1}, n_l, t_k) - 2T(n_i, n_j, n_l, t_k)$$

$$\Delta_k = T(n_i, n_j, n_{l+1}, t_k) + T(n_i, n_j, n_{l-1}, t_k) - 2T(n_i, n_j, n_l, t_k)$$

Un ejemplo más realista

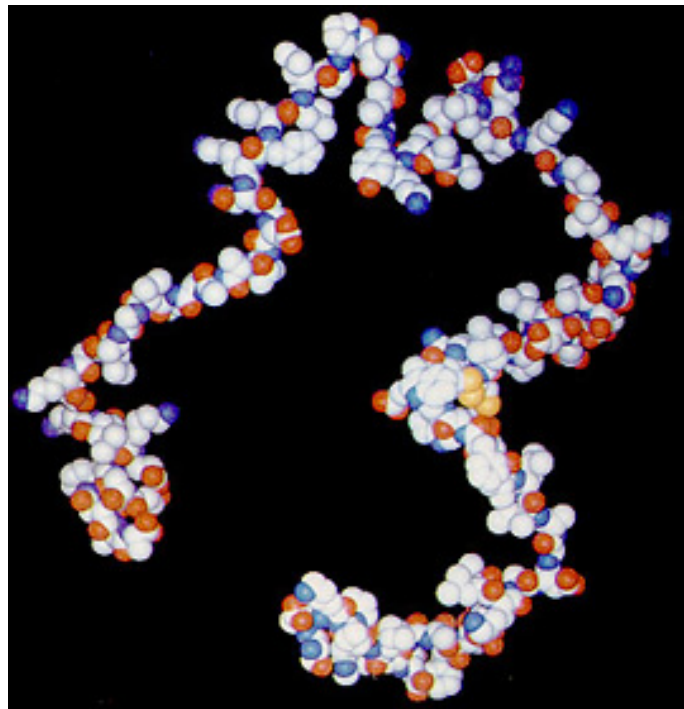
Supongamos que nuestro sistema físico es una cadena de moléculas unidas por enlaces químicos, como por ejemplo una proteína. Podemos pensar en la proteína como unos eslabones unidos fuertemente, pero donde cada eslabón tiene una carga eléctrica (positiva o negativa) y además una cierta afinidad por estar rodeada de otros eslabones o estar lo más aislado posible. Necesitamos calcular las fuerzas que sufre cada eslabón en la cadena, de forma que podamos simular la evolución en el tiempo, con las leyes de Newton ($F=ma$).

Cada eslabón sufre la fuerza Coulombiana de todos los demás eslabones. Esta es una fuerza a larga distancia, es decir, que para calcular la fuerza sobre un eslabón, necesitamos saber la posición de todos los demás eslabones.

Es una interacción No Local, en contraposición con el modelo del Calor anterior, donde es una interacción local: para evolucionar sólo necesitábamos información de sus vecinos, no de los demás.

Si dividimos los eslabones entre procesadores, ahora para calcular las fuerzas, debemos transmitir a cada procesador información de Todos los demás procesadores. Esto supone

un gran aumento del tiempo necesario para la comunicación y una bajada en eficiencia paralela.



Para este tipo de problemas, necesitamos hardware con muy baja latencia y alto ancho de banda.

Otra solución para mejorar el comportamiento, son las máquinas de Memoria compartida, donde en lugar de comunicar entre procesadores pasando mensajes, la memoria RAM es accesible directamente por un gran número de procesadores.

En este caso no es la información la que viaja: los datos están en un lugar fijo, y cada procesador va a buscarlos allá donde se encuentran. La latencia para el acceso a la RAM es muy baja, y el ancho de banda altísimo. Estas máquinas son complejas tecnológicamente y por tanto mucho más caras que las de memoria distribuida.

Alfonso Tarancón Lafita

Estas notas no existirían si no hubiera sido por la ayuda de uno de los mayores expertos en computación de nuestro país, Luis Antonio Fernández, de sabiduría, paciencia y amabilidad casi ilimitados. Para él, mi agradecimiento y admiración.

Ejemplo de código paralelo usando threads

En primer lugar un ejemplo del clásico “hello world”

/

```

*****
* Simple example with pthreads
gcc -lm -lpthread par.c -o par
*****
*****/
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#define NUM_THREADS 5

double result[NUM_THREADS];

void *PrintHello(void *threadid)
{
    long tid;
    double sum=0;
    double x;
    int i,j;
    tid = (long)threadid;
    for(i=0;i<100000;i++)
    {
        for(j=0;j<10;j++)
            x=log(fabs(sin(tid*(double)i/3.1415))+0.1);
        sum+=x;
    }

    result[tid]=sum;
    printf("Hello World! It's me, thread #%ld! with sum=%lf\n",
tid,sum);

    pthread_exit(NULL);
}

int main(int argc, char *argv[])
{
    pthread_t threads[NUM_THREADS];
    int rc;
    long t;
    for(t=0;t<NUM_THREADS;t++){
        printf("In main: creating thread %ld\n", t);
        rc = pthread_create(&threads[t], NULL, PrintHello, (void
*)t);
        if (rc){
            printf("ERROR; return code from pthread_create() is %d\n",
rc);
            exit(-1);

```

```

    }
}

for(t=0;t<NUM_THREADS;t++){
    rc = pthread_join(threads[t], NULL);
    printf("Codigo pthread para t=%d,=%d\n",t,rc);
}

/* Last thing that main() should do */
pthread_exit(NULL);
}

```

Ahora un código que simula N partículas en interacción gravitatoria

```

#include <pthread.h>
#include <math.h>
#include <stdio.h>
//#define DEBUG
double h;
#define N_par 1024
#define NUM_THREADS 4
#define PI 3.14159
double G_Un;
double epsilon;
double M[N_par];
//Compilar : gcc -lpthread -lm -O3 par_g.c -o g

void *Evoluciona_dt(void *P);
void Escribe_resultados(double time,double *x,double *y, double
*z,double *v_x,double *v_y,double *v_z);
double R_INI,V_INI;

double
x[N_par],y[N_par],z[N_par],v_x[N_par],v_y[N_par],v_z[N_par];
double Fx[N_par],Fy[N_par],Fz[N_par];
int N_Proc_Tot;

#define TRUE 1
main()
{

    pthread_t threads[NUM_THREADS];

    double p_cdm_x,p_cdm_y,p_cdm_z,s_m,restar_x,restar_y,restar_z;
    double tiempo;

```

```

int i,j;
long t;
int mesfr;
int rc;

N_Proc_Tot=NUM_THREADS;
G_Un=1000.0;

p_cdm_x=p_cdm_y=p_cdm_z=s_m=0.0;
R_INI=1000;
V_INI=10;

h=0.001; //Paso temporal de la ecuacion diferencial discreta
epsilon=1.0; //Para evitar la singularidad en el cero
tiempo=0;
mesfr=500;

for(i=0;i<N_par;i++)
{
M[i]=1.0;
x[i]=R_INI*sin((double)i/N_par*2*PI);
y[i]=R_INI*cos((double)i/N_par*2*PI);
z[i]=0; // N_par*sin(6.28*i/N_par)+18;
v_x[i]=x[i]*V_INI/R_INI;
v_y[i]=y[i]*V_INI/R_INI;
v_z[i]=0;
p_cdm_x+=M[i]*v_x[i];
p_cdm_y+=M[i]*v_y[i];
p_cdm_z+=M[i]*v_z[i];
s_m+=M[i];
}
restar_x=p_cdm_x/s_m;
restar_y=p_cdm_y/s_m;
restar_z=p_cdm_z/s_m;

for(i=0;i<N_par;i++) //Nos ponemos en el sistema de referencia
centro de masas
{
v_x[i]-=restar_x;
v_y[i]-=restar_y;
v_z[i]-=restar_z;
}

```

```

#ifdef DEBUG
    for(i=0;i<N_par;i++)
        printf("%f          %f          %f          %f          %f\n",x[i],y[i],z[i],v_x[i],v_y[i],v_z[i]);
#endif

    printf("#          t          T          V          E_t \n");

    for(i=0;i<200;i++)
    {
        for(j=0;j<mesfr;j++)
        {
            for(t=0;t<N_Proc_Tot;t++)
                rc = pthread_create(&threads[t], NULL, Evolucion_dt,
(void *)t);

            for(t=0;t<N_Proc_Tot;t++)
                rc = pthread_join(threads[t], NULL);

        }

        tiempo+=mesfr*h;
        Escribe_resultados(tiempo,x,y,z,v_x,v_y,v_z);
    }
}

```

```

void *Evoluciona_dt(void *Proc)
{
    double r2,distance;
    int i,j,i_ini,i_fin;
    long Proc_Id;

    Proc_Id=(long) Proc;

    i_ini=Proc_Id*N_par/N_Proc_Tot;
    i_fin=i_ini+N_par/N_Proc_Tot;
    //printf("particulas=%d,%d\n",i_ini,i_fin);fflush(stdout);

    for(i=i_ini;i<i_fin;i++)
    {
        Fx[i]=Fy[i]=Fz[i]=0.0;
    }
}

```

```

    for(j=0;j<N_par;j++)
    {
        if(i==j)continue;
        r2=epsilon+(x[i]-x[j])*(x[i]-x[j])+(y[i]-y[j])*(y[i]-y[j])+(z[i]-z[j])*(z[i]-z[j]);
        distance=sqrt(r2);
        r2=r2*distance;
        Fx[i] -= (G_Un*M[i]*M[j]*(x[i]-x[j])/r2);
        Fy[i] -= (G_Un*M[i]*M[j]*(y[i]-y[j])/r2);
        Fz[i] -= (G_Un*M[i]*M[j]*(z[i]-z[j])/r2);
    }

    x[i]+=h*v_x[i];
    y[i]+=h*v_y[i];
    z[i]+=h*v_z[i];

    v_x[i] += (Fx[i]*h/M[i]);
    v_y[i] += (Fy[i]*h/M[i]);
    v_z[i] += (Fz[i]*h/M[i]);

}

}

```

```

void Escribir_resultados(double time,double *x,double *y, double
*z,double *v_x,double *v_y,double *v_z)
{
    double r2,distancia;
    double Energia_c,Energia_p,Energia;
    int i,j;

    Energia_p=Energia_c=0.0;

    for(i=0;i<N_par;i++)
    {
        for(j=0;j<N_par;j++)
        {
            if(j==i)continue;

            r2=epsilon+(x[i]-x[j])*(x[i]-x[j])+(y[i]-y[j])*(y[i]-
y[j])+(z[i]-z[j])*(z[i]-z[j]);

```

```

        distancia=sqrt(r2);
        Energia_p-=(G_Un*M[i]*M[j]/distancia/2.0);
        /* Cuento todo dos veces !!! por eso /2*/
    }

    Energia_c+=0.5*M[i]*(v_x[i]*v_x[i]+v_y[i]*v_y[i]
+v_z[i]*v_z[i]);

#ifdef DEBUG
    printf("i=%3d,r=(-10.31f,-10.31f,-10.31f),v=(-10.31f,-10.31f,-10.31f),Ec=-10.31f,Ep=-10.31f\n",
i,x[i],y[i],z[i],v_x[i],v_y[i],v_z[i],Energia_c,Energia_p);
#endif
    }
    Energia=Energia_c+Energia_p;

    printf(" %f %f %f %1f %f %f %f %f %f %f \n",time,
Energia_c,Energia_p, Energia,x[0],y[0],z[0],x[1],y[1],z[1]);
    //getchar();
}

```